

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengenalan Mikroprosesor

Muhammad Sayyid Tsabit - 5024241013

2026

1 Langkah-Langkah Percobaan

1.1 Persiapan

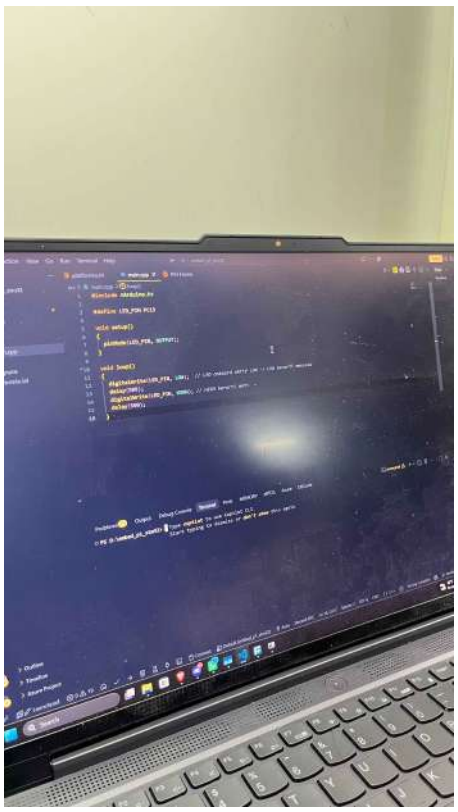
Alat dan bahan yang digunakan adalah satu board STM32 Blackpill (STM32F401CCU6), satu ST-Link V2, satu board ESP32 DevKit v1, empat kabel jumper female-female, dua kabel USB, satu breadboard 830 titik, kabel jumper male-male secukupnya, satu LED dengan resistor $220\ \Omega$, dua pushbutton tactile, tiga resistor $10\ \mathrm{k}\Omega$, satu LDR, dan satu laptop dengan VSCode. Sebelum praktikum dimulai, VSCode, ekstensi PlatformIO, dan driver USB (ST-Link untuk STM32 dan CH340 untuk ESP32) sudah terpasang.

1. Siapkan board STM32 Blackpill + ST-Link, dan board ESP32 DevKit, namun jangan disambungkan terlebih dahulu sebelum instruksi pada masing-masing percobaan.



Gambar 1: Langkah 1 Persiapan: board STM32 Blackpill, ST-Link, dan ESP32 DevKit yang disiapkan

2. Buat satu project PlatformIO untuk seluruh Modul 1 (mis. Name: modul1-praktikum; Board dan Framework saat pembuatan awal bebas dipilih, karena akan diganti lewat platformio.ini sesuai kebutuhan tiap percobaan). Gunakan project yang sama untuk Percobaan 1 sampai 5, jangan membuat project baru di setiap percobaan. Setiap percobaan cukup mengganti isi platformio.ini serta isi src/main.cpp atau src/main.c sesuai kode program yang bersangkutan.



Gambar 2: Langkah 2 Persiapan: project PlatformIO untuk Modul 1

1.2 Percobaan 1: Pengenalan STM32 Blackpill (Blink)

Percobaan ini tidak memerlukan rangkaian tambahan, cukup menggunakan LED onboard (PC13) dan koneksi ST-Link. Setiap kali isi platformio.ini diganti, PlatformIO akan melakukan refresh konfigurasi project. Proses ini butuh beberapa saat, sehingga perlu ditunggu sampai selesai sebelum menekan Build atau Upload.

Listing 1: platformio.ini Percobaan 1

```
[env:blackpill_f401cc]
platform = ststm32
board = blackpill_f401cc
framework = arduino
upload_protocol = stlink
debug_tool = stlink
```

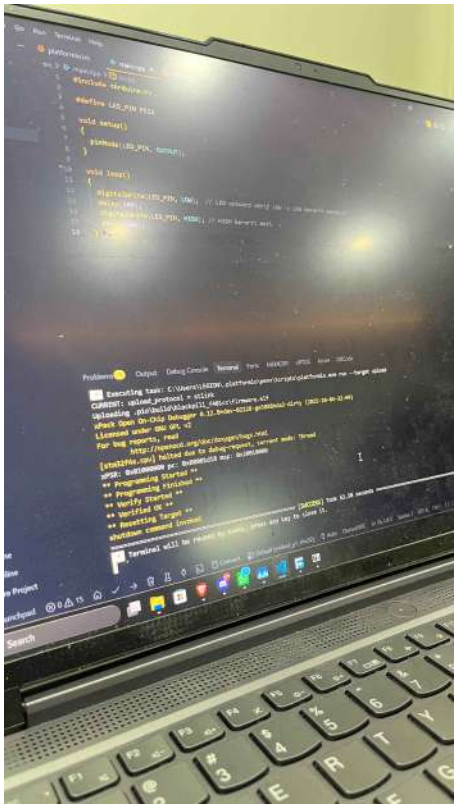
1. Pastikan project Modul 1 sudah dibuat (Bagian Persiapan) dan isi platformio.ini sama seperti di atas.

2. Hubungkan ST-Link ke Blackpill dengan kabel, lalu colokkan ST-Link ke komputer lewat USB. Cocokkan nama pinnya, karena letak pin ST-Link bisa berbeda-beda: SWDIO ke SWDIO (jalur data/debug), SWCLK ke SWCLK (jalur clock/debug), GND ke GND (referensi ground bersama), dan 3.3V ke 3V3 (tegangan target board). Pastikan ST-Link memakai tegangan 3.3V.



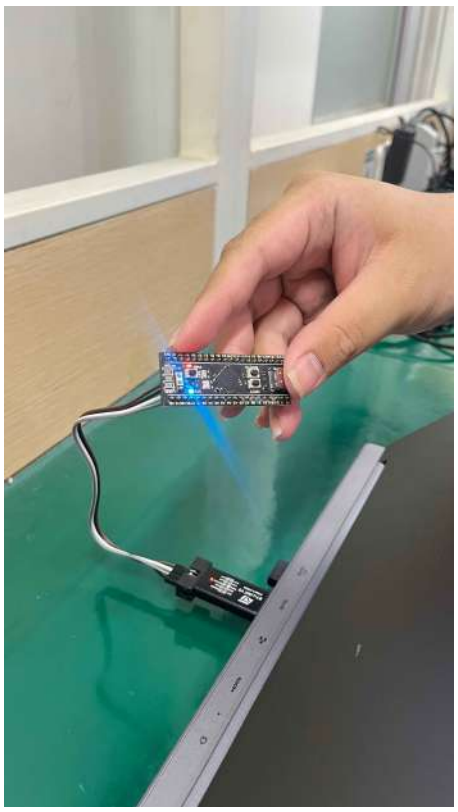
Gambar 3: Langkah 2 Percobaan 1: sambungan ST-Link ke Blackpill

3. Tulis kode Blink di bawah pada src/main.cpp, lalu klik Build dan Upload.



Gambar 4: Langkah 3 Percobaan 1: penulisan kode Blink serta proses Build dan Upload

4. Lihat LED di board (PC13): nyala setengah detik, mati setengah detik, berulang terus.



Gambar 5: Langkah 4 Percobaan 1: LED onboard PC13 menyala dan mati bergantian

Listing 2: Kode Blink STM32 (Arduino)

```
#include <Arduino.h>

#define LED_PIN PC13

void setup()
{
  pinMode(LED_PIN, OUTPUT);
}

void loop()
{
  digitalWrite(LED_PIN, LOW);  // LED onboard aktif LOW -> LOW berarti menyala
  delay(500);
  digitalWrite(LED_PIN, HIGH); // HIGH berarti mati
  delay(500);
}
```

1.3 Percobaan 2: Pengenalan ESP32 dengan Framework ESP-IDF (Blink)

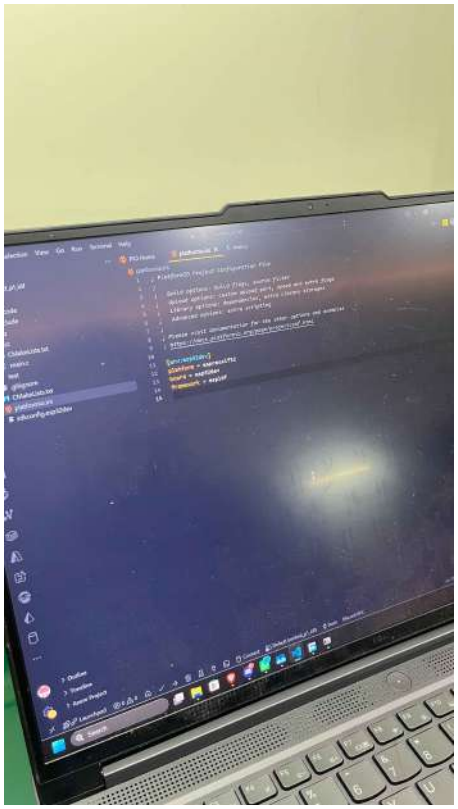
Percobaan ini tidak memerlukan rangkaian tambahan, cukup menggunakan LED onboard ESP32. Seperti pada Percobaan 1, penggantian isi platformio.ini memicu refresh konfigurasi PlatformIO, sehingga perlu ditunggu sampai selesai sebelum Build atau Upload.

Listing 3: platformio.ini Percobaan 2

```
[env:esp32dev]
```

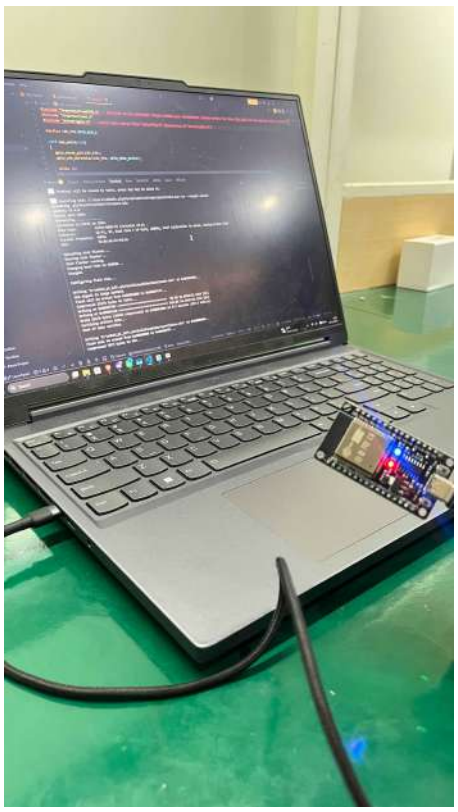
```
platform = espressif32
board = esp32dev
framework = espidf
```

1. Ganti isi platformio.ini dengan konfigurasi di atas (board ESP32 DevKit, framework ESP-IDF).



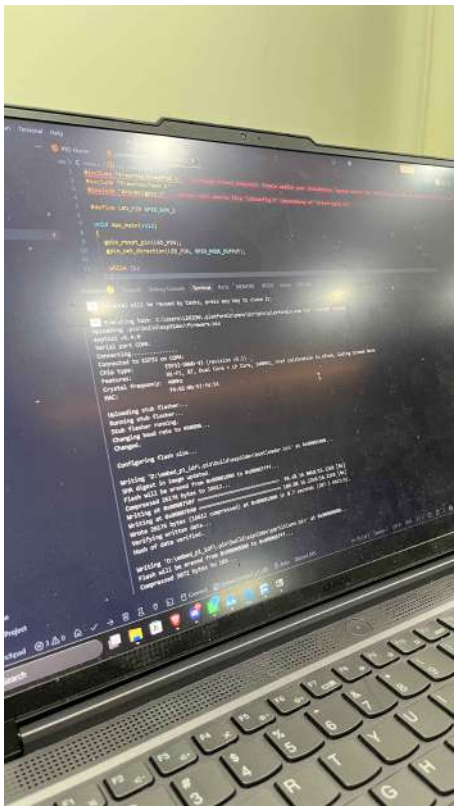
Gambar 6: Langkah 1 Percobaan 2: isi platformio.ini untuk ESP32 dengan framework ESP-IDF

2. Colokkan ESP32 ke komputer lewat USB, lalu pastikan komputer mengenali port-nya.



Gambar 7: Langkah 2 Percobaan 2: ESP32 terhubung ke komputer dan port terdeteksi

3. Hapus file src/main.cpp, buat file baru src/main.c, tulis kode Blink di bawah, lalu klik Build dan Upload.



Gambar 8: Langkah 3 Percobaan 2: kode Blink ESP-IDF pada src/main.c serta proses Build dan Upload

4. Analisis apa perbedaan memprogram dengan ESP-IDF dan dengan Arduino.

Listing 4: Kode Blink ESP32 (ESP-IDF)

```
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/gpio.h"

#define LED_PIN GPIO_NUM_2

void app_main(void)
{
  gpio_reset_pin(LED_PIN);
  gpio_set_direction(LED_PIN, GPIO_MODE_OUTPUT);

  while (1)
  {
    gpio_set_level(LED_PIN, 1);
    vTaskDelay(pdMS_TO_TICKS(500));
    gpio_set_level(LED_PIN, 0);
    vTaskDelay(pdMS_TO_TICKS(500));
  }
}
```

1.4 Percobaan 3: Pull-up dan Pull-down: Eksternal vs Internal (ESP32 + Framework Arduino)

Percobaan ini menggunakan dua pushbutton yang sama untuk kedua kondisi.

Listing 5: platformio.ini Percobaan 3

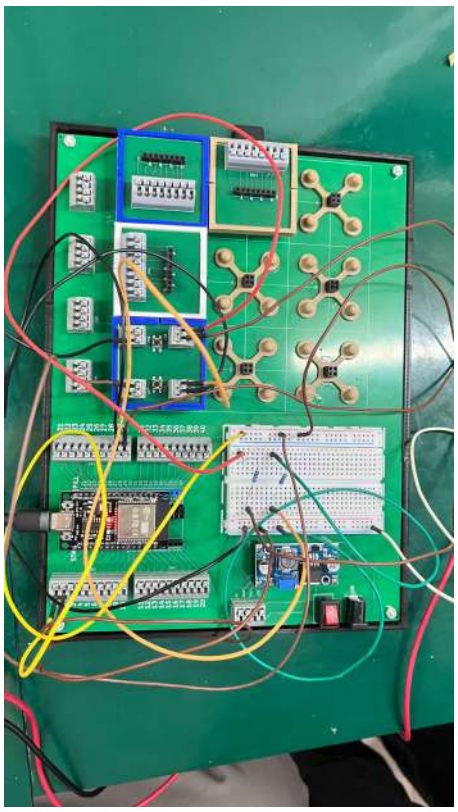
```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
```

Pada Bagian A (pull-up dan pull-down eksternal), pushbutton 1 dipasang pada GPIO 32 dengan satu kaki ke GPIO dan ke 3.3V melalui resistor $10\ \mathrm{k}\Omega$ (pull-up), sedangkan kaki lainnya ke GND. Pushbutton 2 dipasang pada GPIO 33 dengan satu kaki ke GPIO dan ke GND melalui resistor $10\ \mathrm{k}\Omega$ (pull-down), sedangkan kaki lainnya ke 3.3V.

Pada Bagian B (pull-up dan pull-down internal, tanpa resistor), pushbutton 1 pada GPIO 32 dipasang dengan satu kaki ke GPIO dan kaki lainnya ke GND menggunakan INPUT_PULLUP internal. Pushbutton 2 pada GPIO 33 dipasang dengan satu kaki ke GPIO dan kaki lainnya ke 3.3V menggunakan INPUT_PULLDOWN internal. Resistor eksternal dilepas pada bagian ini.

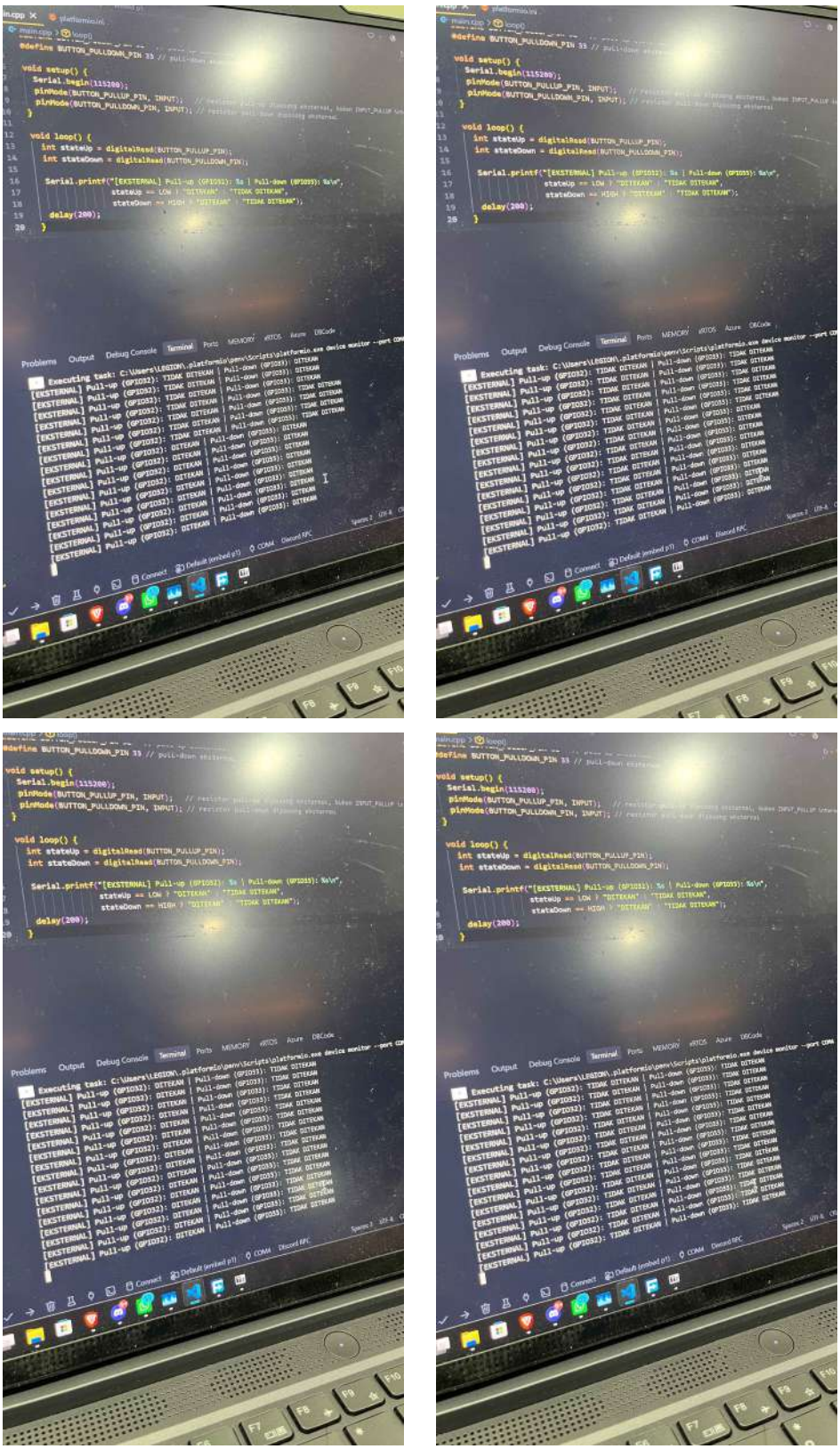
1. Ganti isi platformio.ini kembali ke konfigurasi Arduino di atas. Hapus src/main.c, lalu buat lagi file src/main.cpp.

2. Pasang dua tombol di GPIO 32 dan 33 lengkap dengan resistornya, sesuai skema Bagian A.



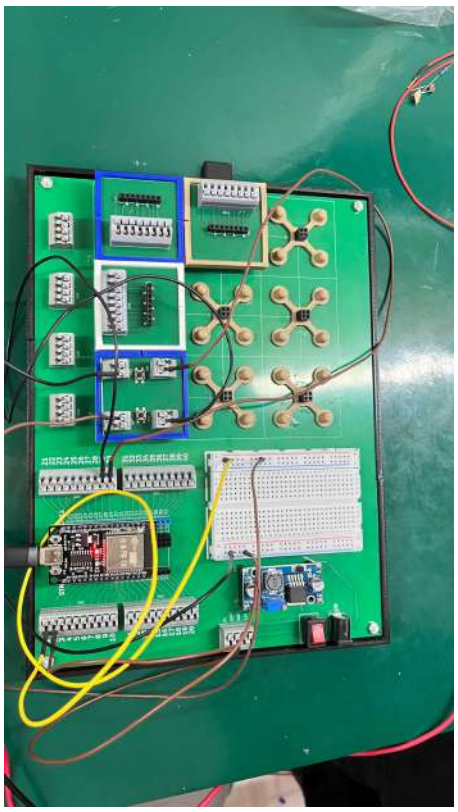
Gambar 9: Langkah 2 Percobaan 3: rangkaian dua pushbutton dengan resistor pull-up dan pull-down eksternal pada GPIO 32 dan 33

3. Tulis Kode Program A (Eksternal) di bawah pada src/main.cpp, klik Build dan Upload, lalu coba tekan kedua tombol dan catat hasilnya dari serial monitor.



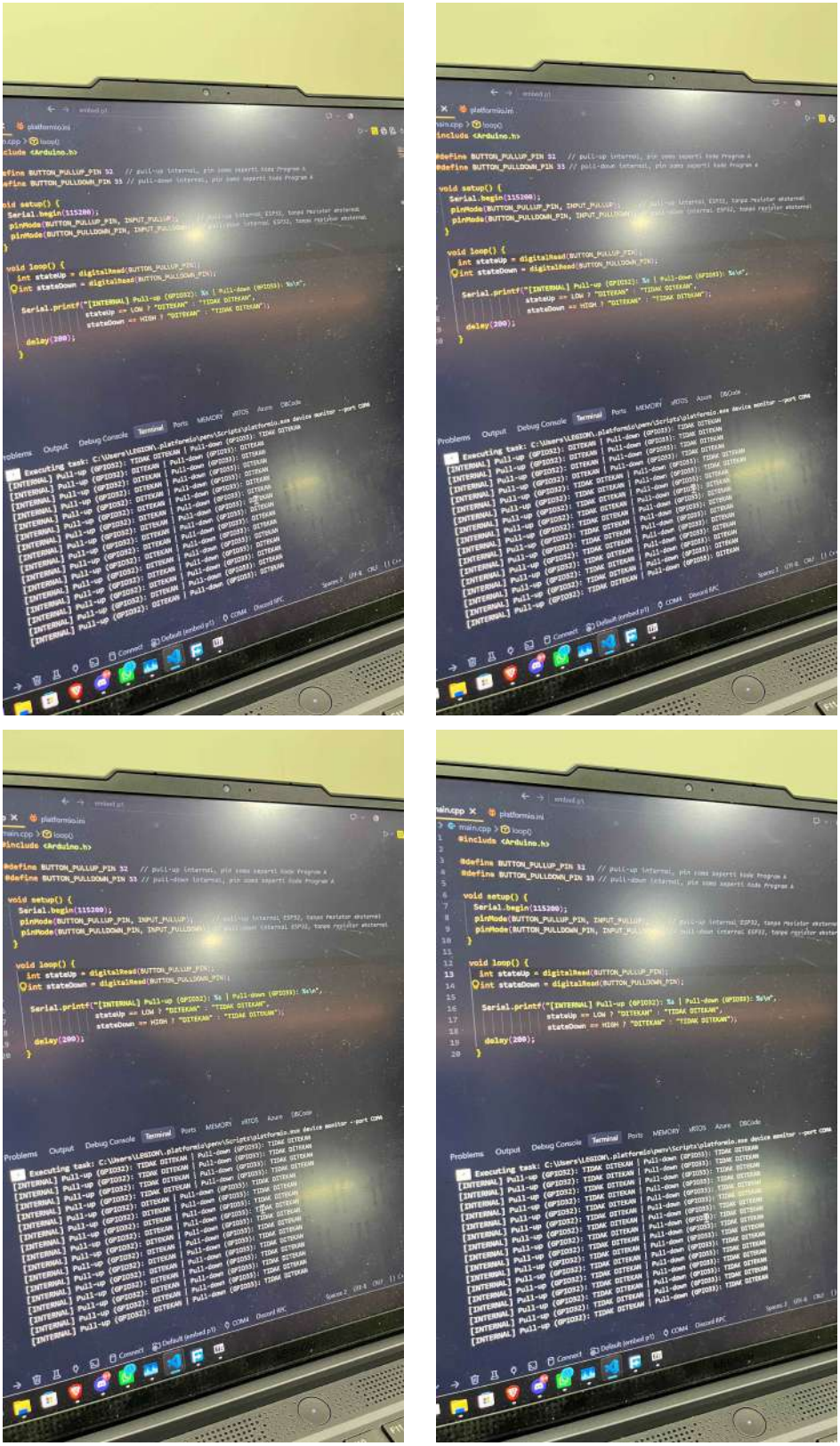
Gambar 10: Langkah 3 Percobaan 3: pengujian Kode Program A (resistor eksternal) dan hasil pembacaan tombol pada Serial Monitor

4. Lepas kedua resistor sesuai skema Bagian B, tombol tetap terpasang. Pastikan pasangan kaki tombol yang benar menurut orientasi kaki pushbutton tactile pada modul.



Gambar 11: Langkah 4 Percobaan 3: rangkaian pushbutton setelah kedua resistor dilepas

5. Ganti isi src/main.cpp dengan Kode Program B (Internal) di bawah, klik Build dan Upload, lalu coba tombol yang sama dan catat hasilnya dari serial monitor.



Gambar 12: Langkah 5 Percobaan 3: pengujian Kode Program B (resistor internal) dan hasil pembacaan tombol pada Serial Monitor

6. Bandingkan hasil A dan B. Seharusnya sama: pull-up bernilai HIGH saat tidak ditekan dan LOW saat ditekan, pull-down kebalikannya.

7. Jika hasilnya terbalik atau berbeda, periksa apakah kedua resistor sudah benar-benar dilepas.

Listing 6: Kode Program A (Pull-up dan Pull-down Eksternal)

```
#include <Arduino.h>

#define BUTTON_PULLUP_PIN 32   // pull-up eksternal
#define BUTTON_PULLDOWN_PIN 33 // pull-down eksternal

void setup() {
  Serial.begin(115200);
  pinMode(BUTTON_PULLUP_PIN, INPUT);   // resistor pull-up dipasang eksternal
  pinMode(BUTTON_PULLDOWN_PIN, INPUT); // resistor pull-down dipasang eksternal
}

void loop() {
  int stateUp = digitalRead(BUTTON_PULLUP_PIN);
  int stateDown = digitalRead(BUTTON_PULLDOWN_PIN);

  Serial.printf("[EKSTERNAL] Pull-up (GPIO32): %s | Pull-down (GPIO33): %s\n",
                stateUp == LOW ? "DITEKAN" : "TIDAK DITEKAN",
                stateDown == HIGH ? "DITEKAN" : "TIDAK DITEKAN");
  delay(200);
}
```

Listing 7: Kode Program B (Pull-up dan Pull-down Internal)

```
#include <Arduino.h>

#define BUTTON_PULLUP_PIN 32   // pull-up internal
#define BUTTON_PULLDOWN_PIN 33 // pull-down internal

void setup() {
  Serial.begin(115200);
  pinMode(BUTTON_PULLUP_PIN, INPUT_PULLUP);     // pull-up internal ESP32
  pinMode(BUTTON_PULLDOWN_PIN, INPUT_PULLDOWN); // pull-down internal ESP32
}

void loop() {
  int stateUp = digitalRead(BUTTON_PULLUP_PIN);
  int stateDown = digitalRead(BUTTON_PULLDOWN_PIN);

  Serial.printf("[INTERNAL] Pull-up (GPIO32): %s | Pull-down (GPIO33): %s\n",
                stateUp == LOW ? "DITEKAN" : "TIDAK DITEKAN",
                stateDown == HIGH ? "DITEKAN" : "TIDAK DITEKAN");
  delay(200);
}
```

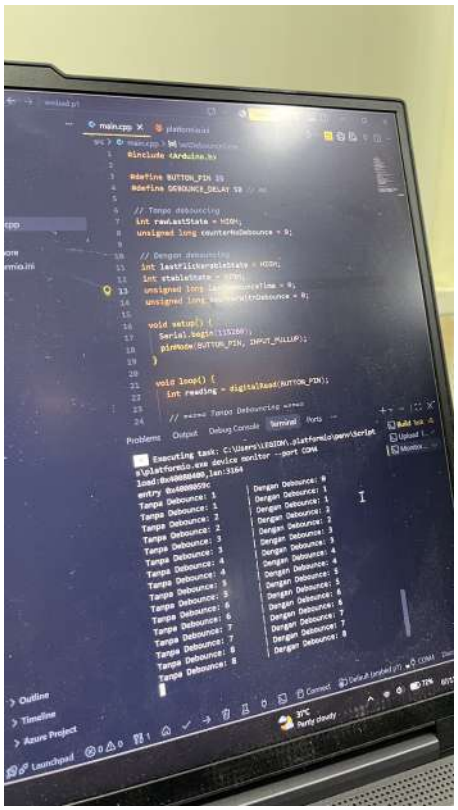
1.5 Percobaan 4: Debouncing pada Input Tombol GPIO (ESP32 + Framework Arduino)

Pada percobaan ini, pushbutton dipasang pada GPIO 25 dengan satu kaki ke GPIO dan kaki lainnya ke GND, menggunakan INPUT_PULLUP. Isi platformio.ini sama dengan Percobaan 3, sehingga langkah penggantian konfigurasi dapat dilewati dan tidak perlu menunggu refresh ulang.

1. Pasang tombol sesuai skema di atas.

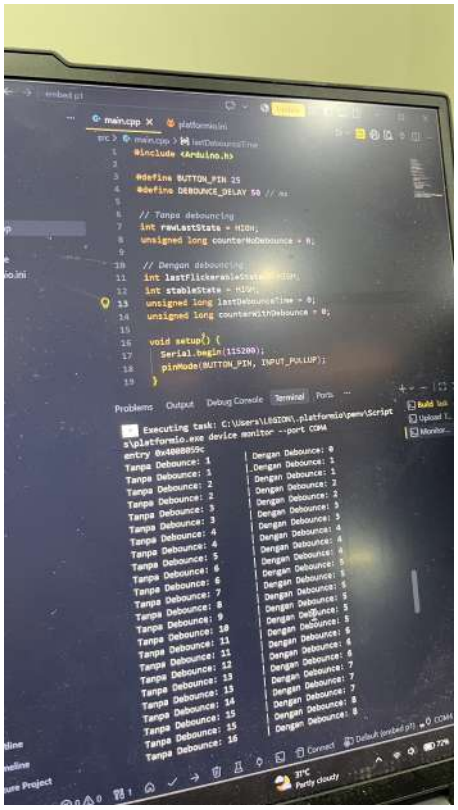
2. Tulis kode di bawah. Kode ini punya dua penghitung: counterNoDebounce (tanpa penyaring getaran tombol) dan counterWithDebounce (dengan penyaring).

3. Tekan tombol satu kali, lalu bandingkan nilai kedua penghitung di Serial Monitor.



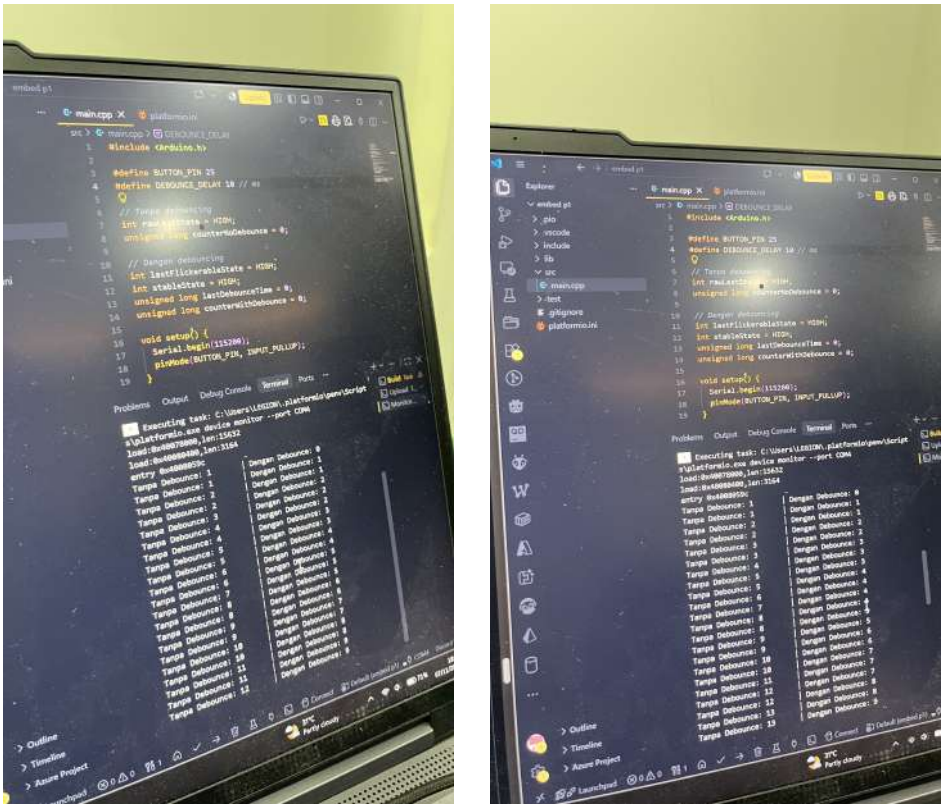
Gambar 13: Langkah 3 Percobaan 4: nilai counterNoDebounce dan counterWithDebounce setelah tombol ditekan satu kali

4. Ulangi dengan cara menekan yang berbeda (cepat dan lambat). counterNoDebounce bisa naik lebih dari 1 untuk satu tekanan, sedangkan counterWithDebounce seharusnya naik tepat 1. Jika keduanya terlihat sama, coba tekan lagi karena getaran tombol tidak selalu muncul.



Gambar 14: Langkah 4 Percobaan 4: nilai kedua penghitung pada penekanan tombol cepat dan lambat

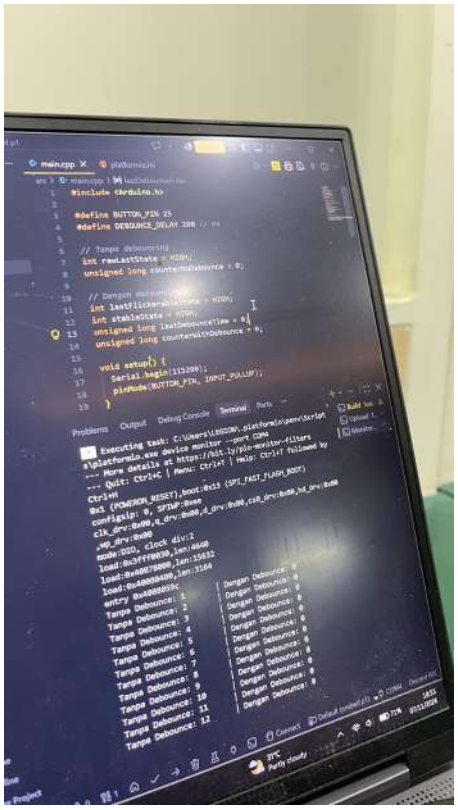
5. Ubah DEBOUNCE_DELAY menjadi 10 (10 ms), klik Build dan Upload, lalu tekan tombol lagi.



Gambar 15: Langkah 5 Percobaan 4: pengujian dengan DEBOUNCE_DELAY 10 ms

6. Ubah menjadi 200 (200 ms), klik Build dan Upload, lalu tekan tombol dengan cepat berulang

kali.



Gambar 16: Langkah 6 Percobaan 4: pengujian dengan DEBOUNCE_DELAY 200 ms

7. Bandingkan ketiga nilai (10, 50, dan 200 ms). Analisis untung ruginya: makin besar nilainya makin bersih dari getaran, tetapi tombol makin lambat merespons.

Listing 8: Kode perbandingan counter tanpa dan dengan debouncing

```
#include <Arduino.h>

#define BUTTON_PIN 25
#define DEBOUNCE_DELAY 50 // ms

// Tanpa debouncing
int rawLastState = HIGH;
unsigned long counterNoDebounce = 0;

// Dengan debouncing
int lastFlickerableState = HIGH;
int stableState = HIGH;
unsigned long lastDebounceTime = 0;
unsigned long counterWithDebounce = 0;

void setup() {
  Serial.begin(115200);
  pinMode(BUTTON_PIN, INPUT_PULLUP);
}

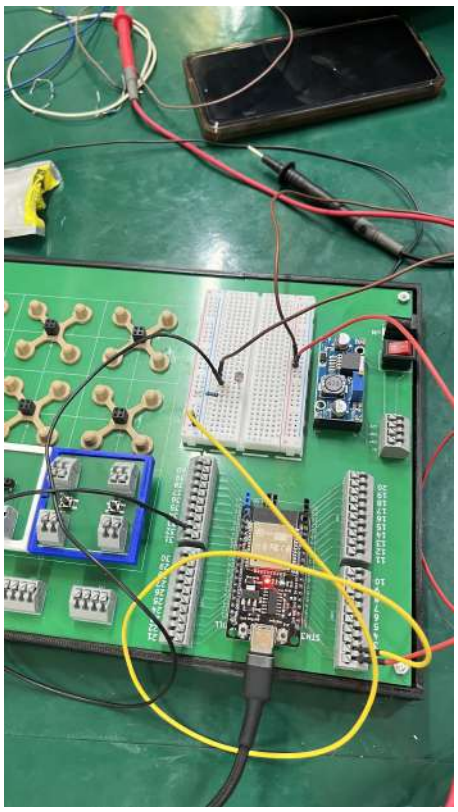
void loop() {
  int reading = digitalRead(BUTTON_PIN);
```

```
23
24    // ===== Tanpa Debouncing =====
25    if (reading == LOW && rawLastState == HIGH) {
26      counterNoDebounce++;
27    }
28    rawLastState = reading;
29
30    // ===== Dengan Debouncing =====
31    if (reading != lastFlickerableState) {
32      lastDebounceTime = millis();
33    }
34    if ((millis() - lastDebounceTime) > DEBOUNCE_DELAY) {
35      if (reading != stableState) {
36        stableState = reading;
37        if (stableState == LOW) {
38          counterWithDebounce++;
39        }
40      }
41    }
42    lastFlickerableState = reading;
43
44    // ===== Tampilkan kedua nilai secara bersamaan =====
45    static unsigned long lastPrintedNoDebounce = 0, lastPrintedWithDebounce = 0;
46    if (counterNoDebounce != lastPrintedNoDebounce ||
47        counterWithDebounce != lastPrintedWithDebounce) {
48      Serial.printf("Tanpa Debounce: %lu\t| Dengan Debounce: %lu\n",
49                    counterNoDebounce, counterWithDebounce);
50      lastPrintedNoDebounce = counterNoDebounce;
51      lastPrintedWithDebounce = counterWithDebounce;
52    }
53  }
```

1.6 Percobaan 5: Akses GPIO Analog: Pembacaan ADC dengan LDR (ESP32 + Framework Arduino)

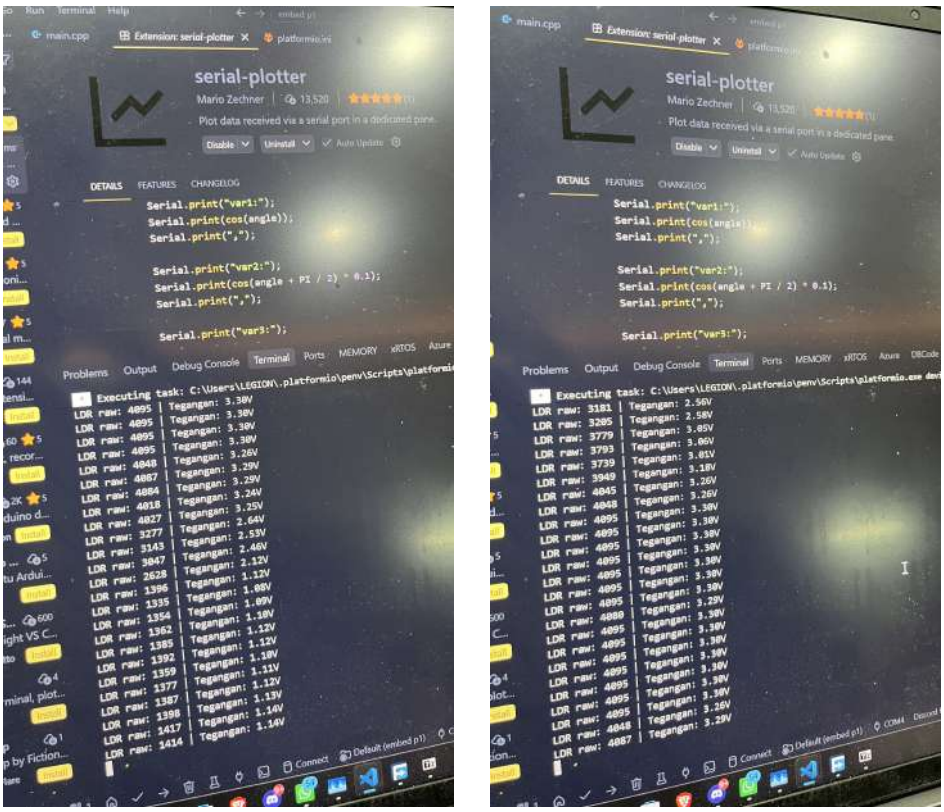
Pada percobaan ini, satu kaki LDR dihubungkan ke 3.3V, sedangkan kaki lainnya dihubungkan ke GPIO 34 dan ke GND melalui resistor $10\ \mathrm{k}\Omega$. Titik tengah pembagi tegangan tersebut menjadi masukan ADC. Isi platformio.ini sama dengan Percobaan 3 dan 4, sehingga langkah penggantian konfigurasi dapat dilewati.

1. Pasang LDR bersama resistor sesuai skema, lalu hubungkan titik tengah keduanya ke GPIO 34.



Gambar 17: Langkah 1 Percobaan 5: rangkaian LDR dan resistor $10\ \mathrm{k}\Omega$ yang terhubung ke GPIO 34

2. Tulis program yang membaca nilai ADC mentah dan mengubahnya menjadi tegangan.
3. Klik Build dan Upload, lalu buka Serial Monitor.
4. Tutup LDR dengan tangan, lalu sinari dengan lampu. Lihat nilai mentahnya (0 sampai 4095) berubah.



Gambar 18: Langkah 4 Percobaan 5: nilai ADC saat LDR ditutup tangan (kiri) dan saat disinari lampu (kanan)

5. Analisis mengapa sinyal analog bisa menunjukkan banyak kondisi, tidak hanya HIGH atau LOW.

Listing 9: Kode pembacaan ADC dengan LDR

```
#include <Arduino.h>

#define LDR_PIN 34

void setup() {
  Serial.begin(115200);
}

void loop() {
  int raw = analogRead(LDR_PIN);
  float voltage = raw * (3.3 / 4095.0);

  Serial.printf("LDR raw: %d | Tegangan: %.2fV\n", raw, voltage);
  delay(300);
}
```

2 Analisis Hasil Percobaan

Secara umum, kelima percobaan pada Modul 1 dapat diselesaikan dengan baik oleh kelompok kami dan tidak ada kendala yang berarti. Pada setiap percobaan, program berhasil di-build dan di-upload, dan hasilnya sesuai dengan yang dijelaskan pada modul. Satu masalah kecil muncul pada Percobaan 3 yang menggunakan rangkaian resistor pull-up/pull-down eksternal, dan dibahas pada bagian Percobaan 3.

2.1 Percobaan 1

LED onboard pada pin PC13 menyala selama setengah detik dan mati selama setengah detik secara berulang, sesuai dengan nilai delay(500) pada kode. LED bawaan Blackpill bersifat aktif LOW, sehingga digitalWrite(LED_PIN, LOW) menyalakan LED dan HIGH mematikannya. Hal ini berkebalikan dengan LED eksternal yang umumnya menyala saat diberi HIGH. Karena Blackpill tidak memiliki programmer bawaan, keberhasilan upload menunjukkan bahwa keempat sambungan antara ST-Link dan Blackpill (SWDIO, SWCLK, GND, dan 3.3V) sudah benar (STMicroelectronics, n.d.).

2.2 Percobaan 2

LED onboard ESP32 berhasil berkedip dengan selang yang sama menggunakan framework ESP-IDF. Dibandingkan dengan Arduino, program dimulai dari app_main(), bukan setup() dan loop(), sehingga pengulangan dibuat sendiri dengan while(1). Pin diatur dengan gpio_reset_pin(), gpio_set_direction(), dan gpio_set_level(), yang masing-masing setara dengan pinMode() dan digitalWrite() pada Arduino. Jeda waktu memakai vTaskDelay(pdMS_TO_TICKS(500)) yang berbasis tick FreeRTOS, sehingga task lain tetap bisa berjalan selama jeda. Kode ESP-IDF juga lebih panjang karena perlu memanggil header FreeRTOS dan driver GPIO. Sebagai gantinya, ESP-IDF memberi akses yang lebih luas ke fitur ESP32 seperti task, semaphore, dan queue (Espressif Systems, n.d.-a).

2.3 Percobaan 3

Pada Bagian A dan Bagian B, hasil pembacaan di Serial Monitor sama. Pada pull-up, tombol terbaca HIGH saat tidak ditekan dan LOW saat ditekan, sedangkan pada pull-down terjadi sebaliknya, sesuai dengan teori. Pada Bagian B, resistor eksternal sudah dilepas dan pembacaan tetap stabil karena INPUT_PULLUP dan INPUT_PULLDOWN mengaktifkan resistor di dalam ESP32. Fitur ini dapat dipakai pada GPIO 32 dan 33, tetapi tidak pada GPIO 34 sampai 39 yang tidak memiliki resistor pull internal (Espressif Systems, n.d.-b).

Masalah yang sempat terjadi ada pada Bagian A. Rangkaian pull-up/pull-down eksternal mengalami short di bagian tombol karena kami kurang cermat membaca sambungan kaki tombol. Pushbutton tactile memiliki empat kaki, dan dua kaki pada sisi yang sama sudah terhubung satu sama lain, sehingga tombol harus dipasang pada pasangan kaki yang berseberangan agar sambungannya hanya terjadi saat tombol ditekan. Kami memeriksa kembali orientasi kaki tombol dan memindahkan sambungannya ke pasangan kaki yang benar. Setelah itu percobaan dapat dijalankan dengan hasil yang sesuai.

2.4 Percobaan 4

Pada percobaan ini, counterWithDebounce bertambah tepat satu untuk setiap penekanan tombol, sesuai dengan tujuan debouncing. Pada counterNoDebounce, penambahan lebih dari satu dapat terjadi karena kontak tombol memantul saat ditekan, tetapi hal ini tidak selalu muncul pada setiap penekanan (isi sesuai pengamatan kelompok, misalnya nilai kedua counter pada beberapa kali penekanan).

Pengaruh nilai DEBOUNCE_DELAY juga diamati. Pada nilai 10 ms, waktu penyaringan singkat sehingga pantulan yang berlangsung lebih lama berpeluang masih terhitung sebagai penekanan baru. Pada nilai 50 ms, pantulan tersaring dan tombol tetap responsif. Pada nilai 200 ms, sinyal sangat bersih dari getaran, tetapi penekanan yang cepat dan berulang dapat tidak terhitung karena sinyal

belum dianggap stabil dalam waktu tersebut. Jadi, makin besar nilai delay, makin bersih hasilnya, tetapi respons tombol makin lambat (isi sesuai pengamatan kelompok).

2.5 Percobaan 5

Nilai ADC berubah mengikuti intensitas cahaya yang mengenai LDR. Saat LDR ditutup tangan, resistansinya besar sehingga tegangan di titik tengah pembagi tegangan kecil dan nilai ADC rendah. Saat disinari lampu, resistansi LDR turun sehingga tegangan di GPIO 34 naik dan nilai ADC makin besar (isi nilai ADC dan tegangan yang diperoleh saat gelap dan terang). Tegangan dihitung dari nilai mentah dengan rumus $V = \text{nilai} \times 3.3/4095$, sesuai dengan resolusi ADC 12 bit pada ESP32.

Sinyal analog dapat menunjukkan banyak kondisi karena ADC membaca besar tegangan, bukan hanya membandingkannya dengan satu batas seperti pada input digital. Dengan 12 bit, ADC membagi rentang 0 sampai 3.3V ke dalam 4096 tingkat nilai, sehingga perubahan kecil pada tegangan tetap menghasilkan nilai yang berbeda (Espressif Systems, n.d.-b). GPIO 34 yang dipakai merupakan pin input-only yang mendukung ADC1, sehingga pembacaan tidak terganggu oleh WiFi.

3 Hasil Tugas Modul

Tugas modul berupa simulasi matrix pushbutton 3x3 yang mengendalikan matrix LED 3x3, dibuat di Wokwi dengan ESP32 dan framework Arduino. Setiap tombol mengendalikan LED pada posisi yang sama, dan tiap penekanan tombol yang valid mengubah status LED tersebut. Baris tombol dihubungkan ke GPIO 16, 17, dan 18 sebagai output. Kolom tombol dihubungkan ke GPIO 19, 21, dan 22 sebagai input dengan INPUT_PULLUP. Sembilan LED masing-masing memakai satu GPIO, yaitu GPIO 13, 14, 25, 26, 27, 32, 33, 4, dan 23. Setiap tombol memiliki debouncing sendiri dengan DEBOUNCE_DELAY 50 ms seperti pada Percobaan 4.

Gambar 19: Rangkaian simulasi matrix pushbutton 3x3 dan matrix LED 3x3

Gambar 20: Kondisi awal simulasi, semua LED mati

Gambar 21: Hasil simulasi saat tombol ditekan, LED pada posisi yang sama menyala

Listing 10: Source code matrix pushbutton 3x3 dengan debouncing

```
#include <Arduino.h>

#define DEBOUNCE_DELAY 50 // ms

const int rowPins[3] = {16, 17, 18};
const int colPins[3] = {19, 21, 22};
const int ledPins[3][3] = {
  {13, 14, 25},
  {26, 27, 32},
  {33, 4, 23}
};
```

```
12
13 int lastReading[3][3];
14 int stableState[3][3];
15 unsigned long lastChangeTime[3][3];
16 bool ledState[3][3];
17
18 void setup() {
19   Serial.begin(115200);
20
21   for (int r = 0; r < 3; r++) {
22     pinMode(rowPins[r], OUTPUT);
23     digitalWrite(rowPins[r], HIGH);
24   }
25   for (int c = 0; c < 3; c++) {
26     pinMode(colPins[c], INPUT_PULLUP);
27   }
28   for (int r = 0; r < 3; r++) {
29     for (int c = 0; c < 3; c++) {
30       pinMode(ledPins[r][c], OUTPUT);
31       digitalWrite(ledPins[r][c], LOW);
32       lastReading[r][c] = HIGH;
33       stableState[r][c] = HIGH;
34       lastChangeTime[r][c] = 0;
35       ledState[r][c] = false;
36     }
37   }
38 }
39
40 void loop() {
41   for (int r = 0; r < 3; r++) {
42     digitalWrite(rowPins[r], LOW);   // aktifkan satu baris
43     delayMicroseconds(50);
44
45     for (int c = 0; c < 3; c++) {
46       int reading = digitalRead(colPins[c]);
47
48       if (reading != lastReading[r][c]) {
49         lastChangeTime[r][c] = millis();
50         lastReading[r][c] = reading;
51       }
52
53       if ((millis() - lastChangeTime[r][c]) > DEBOUNCE_DELAY) {
54         if (reading != stableState[r][c]) {
55           stableState[r][c] = reading;
56           if (stableState[r][c] == LOW) {
57             ledState[r][c] = !ledState[r][c];
58             digitalWrite(ledPins[r][c], ledState[r][c]);
59           }
60         }
61       }
62     }
63
64     digitalWrite(rowPins[r], HIGH);  // nonaktifkan baris
65   }
```

```
66 }
```

Jawaban pertanyaan analisis:

1. Debouncing tetap diperlukan karena bouncing terjadi pada kontak mekanik setiap tombol, berapa pun jumlah tombol yang ditekan. Satu tombol saja sudah memantul selama beberapa milidetik saat ditekan, dan ESP32 membaca pantulan itu sebagai beberapa penekanan. Tanpa debouncing, LED bisa berubah status berkali-kali untuk satu kali tekan.

2. Pin kolom memakai pull-up internal (INPUT_PULLUP), sehingga saat tombol tidak ditekan pin terbaca HIGH, dan menjadi LOW saat tombol ditekan. Pin tidak boleh dibiarkan floating karena nilainya menjadi acak akibat noise, sehingga program bisa membaca tombol ditekan padahal tidak ada yang menyentuhnya.

3. Digital input hanya membedakan HIGH dan LOW berdasarkan batas tegangan, sedangkan ADC mengukur besar tegangan dan mengubahnya menjadi angka. ADC ESP32 memiliki 4096 tingkat nilai pada resolusi 12 bit (Espressif Systems, n.d.-b), sehingga tegangan LDR yang berubah mengikuti cahaya dapat terbaca dalam banyak kondisi.

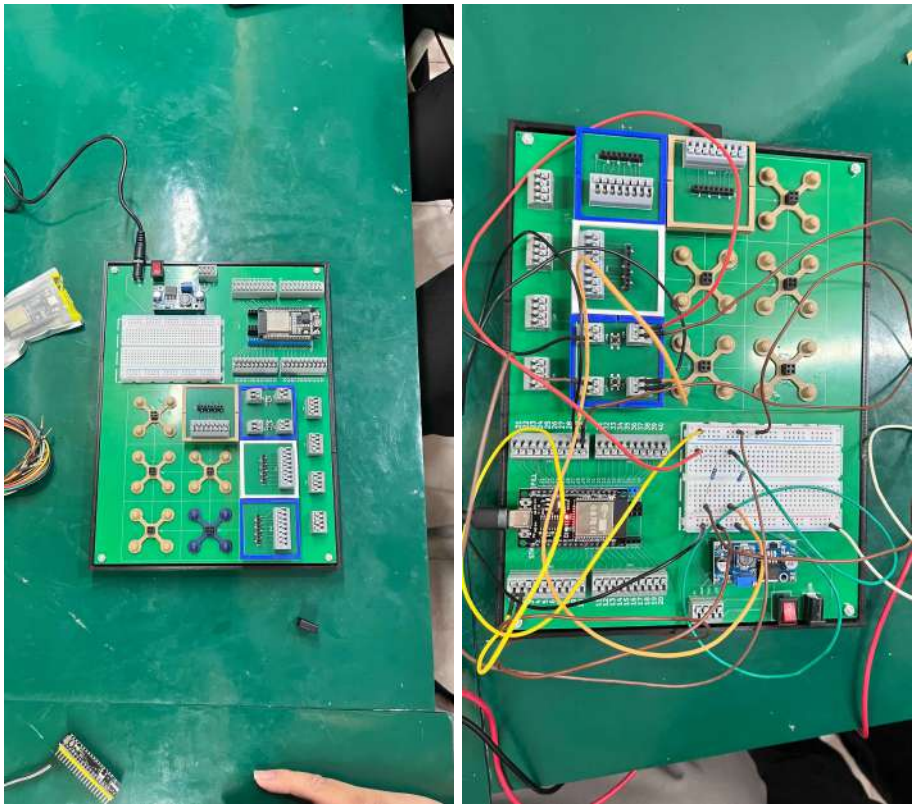
4 Kesimpulan

Praktikum Modul 1 berjalan dengan baik dan kelima percobaan berhasil diselesaikan tanpa kendala yang berarti. PlatformIO pada VSCode berhasil dikonfigurasi dan dipakai untuk memprogram STM32 Blackpill dengan framework Arduino melalui ST-Link, serta ESP32 dengan framework ESP-IDF dan Arduino. Dari Percobaan 1 dan 2 diperoleh pemahaman bahwa Blackpill memerlukan ST-Link karena tidak memiliki programmer bawaan, sedangkan ESP32 dapat diprogram lewat USB. Framework Arduino lebih sederhana, sedangkan ESP-IDF lebih panjang tetapi memberi kendali yang lebih luas.

Pada Percobaan 3, pembacaan tombol dengan pull-up dan pull-down, baik eksternal maupun internal, menghasilkan logika yang sesuai dengan teori. Short pada rangkaian tombol yang sempat terjadi disebabkan kurang cermatnya pembacaan sambungan kaki tombol, dan teratasi setelah sambungan diperiksa kembali. Percobaan 4 menunjukkan bahwa debouncing diperlukan agar satu penekanan terbaca satu kali, dengan nilai delay yang perlu dipilih secara seimbang antara kebersihan sinyal dan kecepatan respons. Percobaan 5 menunjukkan bahwa GPIO dapat dipakai sebagai input analog melalui ADC untuk membaca besaran yang berubah kontinu seperti intensitas cahaya. Tugas modul berupa matrix pushbutton 3x3 dengan debouncing juga berhasil dibuat pada simulasi.

5 Lampiran

5.1 Dokumentasi saat praktikum



Gambar 22: Alat yang digunakan pada praktikum